2026-2027 ACADEMIC YEAR

WEB SERVICES

LAB PRACTICE 2 GUIDE AND EXPERIMENTS

INSTRUCTIONS TO STUDENTS FOR PREPARING A LAB REPORT

This Lab Manual is prepared to help the students with their practical understanding and development of skills, and is used as a base reference during the lab/practical classes. Students have to submit Lab experiment report of this lab practice at 4:30 PM /11:30AM AND at 1:30 PM/8:30AM for previous lab into corresponding next lab, and the lab report to be submitted should include at least the following topics listed clearly in documentation (comments in your program code) at the beginning of each file:

1. **GROUP NAME**
2. **GROUP MEMBERS NAME AND ID**
3. **Title of the program**
4. **Output (compilation)**

All codes must compile. Code that does not compile will not be graded.

GUIDE:

- # PRACTICE REST IN PHP

There are multiple techniques and libraries available with PHP to deal with HTTP. In this practice, our main interest would be on the mechanisms of dealing with HTTP that would help us use REST.

If you host a PHP script with a web server that becomes a resource as per the principles of REST architectural style, then you have a service. So, when you use a web server, you have the luxury of the web server dealing with the HTTP protocol for you. You have little to worry about other than being sensitive to the elements of the protocol such as the verbs in use.

If you are implementing service clients, file_get_contents can be used to access a resource over the HTTP protocol. See the code shown below:

I. REMOTE SERVER AND REMOTE CLIENT COMMUNICATING THROUGH REST ARCHITECTURAL PRINCIPLE (I.E., THROUGH URL)

remoteclient.php

```
a. <?php
b.
c. $options = array(
d.   'http'=>array(
e.     'method'=>"POST",
f.     'header'=>
g.       "Accept-language: en\r\n".
h.       "Content-type: application/x-www-form-
   urlencoded\r\n",
i.
   'content'=>http_build_query(array('foo'=>'bar'
   ))
j. ));
k.
l. $context = stream_context_create($options);
m.
n. //fopen('http://localhost/webservices-
   lab/remoteserver.php',false,$context);
o.
p. $result =
   file_get_contents('http://localhost/webservice
   s-lab/remoteserver.php', NULL, $context);
q.
r. if ( $result !==null)
```

```
s.
t. echo  $result ;
u.
v.
w. ?>
```

remoteserver.php

```
a. <?php
b.
c. function response(){
d.           echo " THE CLIENT IS REMOTELLY
  CONNECTED TO CALL REMOTE PROCEDURE"."<BR>";
e.
f.
g. if(empty($_POST))
h. {
i. echo " THE CLIENT IS REMOTELLY CONNECTED BUT
  NO INPUT DATA "."<BR>";
j.
k. }
l. Else
m. {
n.     echo "input data sent is : "."<BR>" .
  filter_input(INPUT_POST,
o.          "foo");
p.
q. }
r.
s. }
t. response();
u.
v. ?>
```

OUTPUT AFTER TRYING:
http://localhost/webservices-lab/remoteclient.php

THE CLIENT IS REMOTELLY CONNECTED TO CALL REMOTE PROCEDURE
input data sent is :
bar

ALTERNATIVELY

USING CMD AND CURL
cd C:\xampp\htdocs\webservices-lab

C:\xampp\htdocs\webservices-lab>curl --request POST --data "foo=>bar" http://localhost/webservices-lab/remoteserver.php

OUTPUT IS

THE CLIENT IS REMOTELLY CONNECTED TO CALL REMOTE PROCEDURE
input data sent is :
>bar

II. use the CURL PHP API

While using the file_get_contents method would be simple, there are a few limitations in using this function to access service. As an example, this function would always use HTTP GET method on the given URL. You have no control over the HTTP method to be used with this function. Also note that file_put_contents does not support writing to network connections, hence you would not be able to

perform a PUToperation on a resource with that function.
So we need a more feature-packed HTTP client library. We can use the CURL PHP API.

CURL
CURL is an abbreviation for "Client URL Request Library", or sometimes the recursive version "Curl URL Request Library". CURL is a powerful library because of the power of the library that it wraps, namely libcurl. The reasons for libcurl to be considered a powerful library include the fact that it currently supports the http, https, ftp, gopher, telnet, dict, file, and ldap protocols, its support for HTTPS certificates, HTTP POST, HTTP PUT, FTP uploading, HTTP form-based upload, proxies, cookies, and username password authentication. CURL PHP API comes with a wide array of options and features. This allows users to fine tune the requests and the way that the responses are handled.
In this section, we will explore how we can utilize CURL to use various HTTP methods that we would require to consume REST services. Note that CURL is a PHP extension. Usually it comes pre-built with the binary distributions.
There are four main steps when you are using CURL:
1. Initialize CURL
2. Set options
3. Execute CURL
4. Close CURL
These steps, initializing, executing, and closing are standard steps that you would

use irrespective of the HTTP method that you want to use with the URL. It is the set
of options that you have to change based on the HTTP method you want to use. As
an example, CURLOPT_GET option would be used for HTTP GET and CURLOPT_POST
would be used for HTTP POST.
Initialize:
$ch = curl_init();
Set Options:
curl_setopt($ch, CURLOPT_URL, $url);
curl_setopt($ch, CURLOPT_GET, true);
Execute:
curl_exec($ch);
Close:
curl_close($ch);
In the following example (example II), we are using HTTP GET method. We tell CURL to use GET method by setting CURLOPT_GET option to true.

HTTP GET

It is useful to know what to expect when you are using various HTTP verbs. This is because you can always look into the sent message and verify that the correct HTTP verb was used along with parameters.

Also, it is worth mentioning that this script would print the response from the service directly to the console, even though we are not using an echo in the script.
This is because we need one more option to instruct CURL not to send the received data to the standard output. That option is named CURLOPT_RETURNTRANSFER

and must be set to true to get the result of the CURL invocation as a string to a variable.
When using CURL for REST clients, more often than not, you would want to capture the return value to a variable rather than printing it directly as an output, because you often would want to process the response from the service before presenting that to the user.
Here is the same client code with option CURLOPT_RETURNTRANSFER set to true. Note
that unlike in the previous sample, we now have to use echo to display the returned XML response, and in order to echo we need to capture the response returned as a string from the curl_exec function.

Client_CURL.php

```
<?php

$url =
'http://localhost/rest/02/message_trace.php';

$data = <<<XML

<text>Hello World!</text>

XML;

$ch = curl_init();

curl_setopt($ch, CURLOPT_URL, $url);
```

```
curl_setopt($ch, CURLOPT_RETURNTRANSFER,
true);

curl_setopt($ch, CURLOPT_POST, true);

curl_setopt($ch, CURLOPT_POSTFIELDS, $data);

$response = curl_exec($ch);

curl_close($ch);

echo $response;

?>
```

Message_trace.php

```
<?php

$input = file_get_contents("php://input");

file_put_contents("php://output", $input);

?>
```

ON THE BROWSER WINDOW:

http://localhost/rest/02/client_CURL.php

OUTPUT IS

Hello World!

III. Creating a REST Web Service

The easiest Web services to create are those where clients simply make requests. Because no data modifications are taking place, you need to handle only the GET method. For example, suppose you want to write a service that simply adds two integers passed by the client. This is not a very useful service, but it demonstrates the functionality nonetheless. Say you can access this fictitious service at localhost/webservices-lab/add/REST-ADDIT-SERVICE.php/addit. It requires two parameters, num1 and num2. it returns a value element containing the results of the addition.
Here's the code:

/*write the code and save it as REST-ADDIT-SERVICE.php and put it in your web server (htdocs folfer in xampp)/webservices-lab/add/*/

```
<?php

/*REST-ADDIT-SERVICE.php*/

function generate_error($messages) {
$error = '<error>';
/* A message does not contain any characters invalid for element content */
foreach ($messages AS $message) {
$error .= '<message>'.$message.'</message>';
}
$error .= '</error>';
return $error;
}
function addit($num1, $num2) {
$retval = '<value>';
$retval .= $num1 + $num2;
$retval .= '</value>';
return $retval;
}
/* Set content type for XML */
```

```
header('Content-type: text/xml');
print '<?xml version="1.0"?>';
$errors = array();
print addit((int)$_GET['num1'], (int)$_GET['num2']);
if (!isset($_GET['num2'])) {
$errors[] = 'Missing num2 parameter';

print generate_error($errors);
}
?>
```

Calling this service using the URL

localhost/webservices-lab/add/REST-ADDIT-SERVICE.php/addit?num1=1&num2=2

results in the following data:

```
<value>3</value>
```

IV. Create and Consume Simple REST API in PHP

What is REST?

REST stands for Representational State Transfer, REST is an architectural style which defines a set of constraints for developing and consuming web services through standard protocol (HTTP). REST API is a simple, easy to implement and stateless web service. There is another web service available which is SOAP which stands for Simple Object Access Protocol which is created by Microsoft.

REST API is widely used in web and mobile applications as compared to SOAP. REST can provide output data in multiple formats such as JavaScript Object Notation (JSON), Extensible Markup Language (XML), Command Separated Value (CSV) and many others while SOAP described output in Web Services Description Language (WSDL).

How Does REST API Work

REST requests are related to CRUD operations (Create, Read, Update, Delete) in database, REST uses GET, POST, PUT and DELETE requests. Let me compare them with CRUD.

- **GET** is used to retrieve information which is similar to **Read**
- **POST** is used to create new record which is similar to **Create**

- **PUT** is used to update record which is similar to **Update**
- **DELETE** is used to delete record which is similar to **Delete**

How to Create Simple REST API in PHP

JSON format is the most common output format of REST API, we will use the JSON format to consume our simple REST API. We will developed an online transaction payment REST API for our example. I will try to keep it as simple as possible so i will use **GET** request to retrieve information.

1. Create REST API in PHP
2. Consume REST API in PHP

1. Create REST API in PHP

To create a REST API, follow these steps:

1. Create a Database and Table with Dummy Data
2. Create a Database Connection
3. Create a REST API File

1. Create a Database and Table with Dummy Data

To create database run the following query.

```
CREATE DATABASE allphptricks;
```

To create a table run the following query. **Note:** I have already attached the SQL file of this table with dummy data, just download the complete zip file of this tutorial.

```
CREATE TABLE IF NOT EXISTS `transactions` (

`id` int(20) NOT NULL AUTO_INCREMENT,

`order_id` int(50) NOT NULL,

`amount` decimal(9,2) NOT NULL,

`response_code` int(10) NOT NULL,

`response_desc` varchar(50) NOT NULL,
```

```
PRIMARY KEY (`id`),

UNIQUE KEY `order_id` (`order_id`)

) ENGINE=InnoDB DEFAULT CHARSET=latin1 ;
```

2. Create a Database Connection

Just create a **db.php** file and paste the following database connection in it. Make sure that you update these credentials with your database credentials.

```
// Enter your Host, username, password, database below.

$con = mysqli_connect("localhost","root","","allphptricks");

    if (mysqli_connect_errno()){

       echo "Failed to connect to MySQL: " . mysqli_connect_error();

       die();

       }
```

3. Create a REST API File

Create a **api.php** file and paste the following script in it.

```
<?php

header("Content-Type:application/json");

if (isset($_GET['order_id']) && $_GET['order_id']!="") {

       include('db.php');

       $order_id = $_GET['order_id'];

       $result = mysqli_query(

       $con,
```

```
        "SELECT * FROM `transactions` WHERE order_id=$order_id");
        if(mysqli_num_rows($result)>0){
        $row = mysqli_fetch_array($result);
        $amount = $row['amount'];
        $response_code = $row['response_code'];
        $response_desc = $row['response_desc'];
        response($order_id, $amount, $response_code,$response_desc);
        mysqli_close($con);
        }else{
                response(NULL, NULL, 200,"No Record Found");
                }
}else{
        response(NULL, NULL, 400,"Invalid Request");
        }


function response($order_id,$amount,$response_code,$response_desc){
        $response['order_id'] = $order_id;
        $response['amount'] = $amount;
        $response['response_code'] = $response_code;
        $response['response_desc'] = $response_desc;
```

```
        $json_response = json_encode($response);

        echo $json_response;

}

?>
```

The above script will accept the GET request and return output in the JSON format.

Above URL is not user friendly, therefore we will rewrite URL through the .htaccess file, copy paste the following rule in .htaccess file.

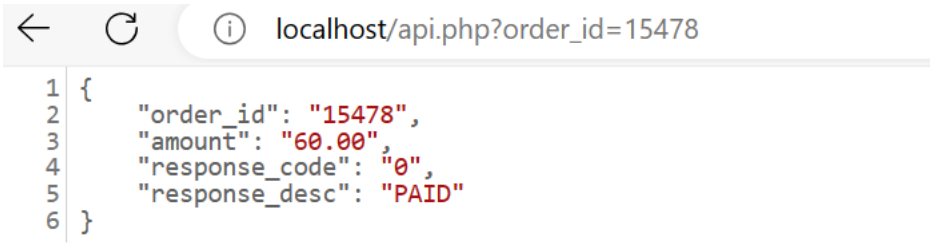
```
RewriteEngine On    # Turn on the rewriting engine

RewriteRule ^api/([0-9a-zA-Z_-]*)$ api.php?order_id=$1 [NC,L]
```

Now you can get the transaction information by browsing the following URL.

localhost /webservices-lab/api/api.php?order_id=15478

You will get the following output.



For the dummy data,

INSERT INTO transactions(order_id, amount, response_code, response_desc)

VALUES ('15478', '60', '0', 'PAID');

V. JAVA RESTFUL EXAMPLE

[I WILL ATTACH THE SOURCE CODE FOR THIS ONE IN ZIP FILE. READ THE README FILES.]

The implementation consists of a JSP (Java Server Pages) script and two backend JavaBeans that the JSP script uses to get the data returned to the client (see Figure 1-6). The data is composed of sage corporate predictions. Here is a sample:

> Decentralized 24/7 hub will target robust web-readiness.
>
> Synergistic disintermediate policy will expedite backend experiences.
>
> Universal fault-tolerant architecture will synthesize bleeding-edge channels.

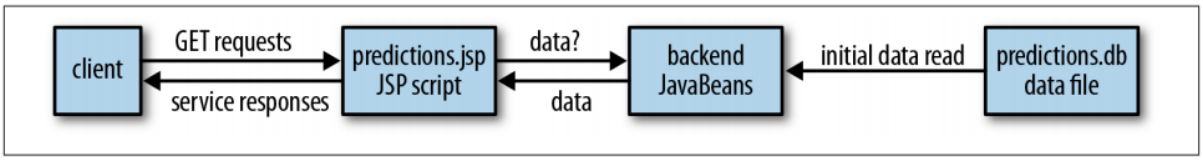


Figure 1-6. The organization of the predictions web service

There is an Ant script that automates the deployment of this and other service examples. Here is a summary of the service parts and how the Ant script puts the parts together:

1. The service consists of a JSP script together with two POJO (JavaBean) classes. The classes provide backend support for the JSP script. There is also a small configuration file, web.xml, that allows the URI to be shortened from: /predictions/predictions.jsp

to: /predictions/

2. The Ant script compiles the .java files and then packages the JSP script, the compiled .class files, and—only for convenience—the .java files into a JAR file with a .war extension (hereafter, a WAR file).

3. The WAR file is copied to the Tomcat webapps subdirectory, which thereby deploys the service. .In the predictions service, each prediction has an associated human predictor. The RESTful resource is thus a list of predictor names (e.g., Hollis McCullough) and their predictions (Hollis is responsible for the third prediction shown above). The resource name or URI is /predictions/, and the only allowable HTTP verb is GET, which corresponds to read among the CRUD operations. If the HTTP request is correct, the RESTful service returns an XML representation of the predictor/prediction list; other-wise, the service returns the appropriate HTTP status code (e.g., 404 for “Not Found” if the URI is incorrect or 405 for “Method Not

Allowed" if the verb is not GET). Example 1-5 shows a slice of the XML payload returned upon a successful request.

Example 1-5. The XML response from the predictions service

```
<?xml version="1.0" encoding="UTF-8"?>

<java version="1.7.0" class="java.beans.XMLDecoder">

<array class="predictions.Prediction" length="32">

 <void index="0">

 <object class="predictions.Prediction">

 <void property="what">

 <string>

 Managed holistic contingency will grow killer action-items.

 </string>

 </void>

 <void property="who">

 <string>Cornelius Tillman</string>

 </void>

 </object>

 </void>

 ...

 <void index="31">

 <object class="predictions.Prediction">

 <void property="what">

 <string>

 Versatile tangible application will maximize rich ebusiness.
```

```
</string>

</void>

<void property="who">

<string>Hiram Gulgowski</string>

</void>

</object>

</void>

</array>

</java>
```

How the Predictions Web Service Works

When the predictions service is deployed to a web server such as Tomcat, the server translates the JSP script predictions.jsp (see Example 1-6) into a servlet instance. For now, this technical detail is overlooked because it is convenient to talk about the JSP script itself as the target of a request.

Example 1-6. The JSP script predictions.jsp

```
<jsp:useBean id = "preds"

type = "predictions.Predictions"

class = "predictions.Predictions">
<% // Check the HTTP verb: if it's anything but GET,

// return 405 (Method Not Allowed).

String verb = request.getMethod();

if (!verb.equalsIgnoreCase("GET")) {

response.sendError(response.SC_METHOD_NOT_ALLOWED,

"Only GET requests are allowed.");

}

// If it's a GET request, return the predictions.
```

```
 else {
 preds.setServletContext(application);
 out.println(preds.getPredictions());
 }
 %>
</jsp:useBean>
```

The backend code consists of two POJO classes, Prediction (see Example 1-7) and Predictions (see Example 1-8). The Prediction class is quite simple, consisting of two properties: who (line 1) is the person making the prediction and what (line 2) is the prediction.

Example 1-7. The backend predictions.Prediction class

```
package predictions;
import java.io.Serializable;
public class Prediction implements Serializable {
 private String who; // person
 private String what; // his/her prediction
 public Prediction() { }
 public void setWho(String who) { this.who = who; }
 public String getWho() { return this.who; }
 public void setWhat(String what) { this.what = what; }
 public String getWhat() { return this.what; }
}
```

The Predictions class does the grunt work. For example, its populate method (line 3) reads the prediction data from a text file, predictions.db, encapsulated in the deployed WAR file, and the toXML method serializes a Java List<Prediction> into an XML document, which is sent back to the client. If there were problems reading or formatting the data, the predictions service would return null to the client.

Example 1-8. The backend predictions.Predictions class

Example 1-8. The backend predictions.Predictions class

```
package predictions;

import java.io.IOException;

import java.io.InputStream;

import java.io.InputStreamReader;

import java.io.BufferedReader;

import java.io.ByteArrayOutputStream;

import java.beans.XMLEncoder; // simple and effective

import javax.servlet.ServletContext;

public class Predictions {

 private int n = 32;

 private Prediction[ ] predictions;

 private ServletContext sctx;

 public Predictions() { }

 public void setServletContext(ServletContext sctx) { this.sctx = sctx; }

 public ServletContext getServletContext() { return this.sctx; }

 // getPredictions returns an XML representation of

 // the Predictions array

 public void setPredictions(String ps) { } // no-op

 public String getPredictions() {

 // Has the ServletContext been set?

 if (getServletContext() == null) return null;

 // Has the data been read already?
```

```
if (predictions == null) populate();

// Convert the Predictions array into an XML document

return toXML();

}

private void populate() {

String filename = "/WEB-INF/data/predictions.db"; InputStream in =
sctx.getResourceAsStream(filename);

// Read the data into the array of Predictions.

if (in != null) {

try {

InputStreamReader isr = new InputStreamReader(in);

BufferedReader reader = new BufferedReader(isr);

predictions = new Prediction[n];

int i = 0;

String record = null;

while ((record = reader.readLine()) != null) {

String[] parts = record.split("!");

Prediction p = new Prediction();

p.setWho(parts[0]);

p.setWhat(parts[1]);

predictions[i++] = p;

}

}

catch (IOException e) { }
```

```
}

}

private String toXML() {

String xml = null;

try {

ByteArrayOutputStream out = new ByteArrayOutputStream();

XMLEncoder encoder = new XMLEncoder(out);

encoder.writeObject(predictions); // serialize to XML

encoder.close();

xml = out.toString(); // stringify

}

catch(Exception e) { }

return xml;

}

}
```

On a GET request, the JSP script invokes the method setServletContext (line 1 in the bean, line 2 in the JSP script) with an argument, the implicit object reference named application. The backend bean needs access to the servlet context in order to read data from a text file embedded in the deployed WAR file. The ServletContext is a data structure through which a servlet/JSP script can interact explicitly with the servlet container. The call to the setServletContext method sets up the subsequent call to the getPredictions method (line 2), which returns the XML representation shown in Example 1-5.

The predictions service can be deployed under the Tomcat web server using a provided Ant script (with % as the command-line prompt):

% ant -Dwar.name=predictions deploy

C:\tomcat50-jwsdp\webapps\jspService>**ant -Dwar.name=app1 deploy**

A Client Against the Predictions Web Service

RESTful clients in Java and other languages; are either a browser or a utility such as curl is good enough. On a successful curl request to the service:

% curl -v http://localhost:8080/predictions/

C:\tomcat50-jwsdp\webapps\jspService>**curl http://localhost:8080/app1/**

OUTPUT

```
<?xml version="1.0" encoding="UTF-8"?>
<java version="1.5.0_22" class="java.beans.XMLDecoder">
 <array class="predictions.Prediction" length="32">
  <void index="0">
   <object class="predictions.Prediction">
    <void property="what">
     <string>Managed holistic contingency will grow killer action-items.</string>
    </void>
    <void property="who">
     <string>Cornelius Tillman</string>
    </void>
   </object>
  </void>
  <void index="1">
   <object class="predictions.Prediction">
    <void property="what">
```

```
    <string>Vision-oriented zero administration time-frame will generate back-end
interfaces.</string>
   </void>
   <void property="who">
    <string>Conner Kulas</string>
   </void>
  </object>
 </void>
 <void index="2">
  <object class="predictions.Prediction">
   <void property="what">
    <string>Triple-buffered scalable function will productize visionary infomediaries.</string>
   </void>
   <void property="who">
    <string>Loraine Ryan</string>
   </void>
  </object>
 </void>
 <void index="3">
  <object class="predictions.Prediction">
   <void property="what">
    <string>Reactive radical knowledge base will aggregate extensible vortals.</string>
   </void>
   <void property="who">
```

```
    <string>Patricia Zulauf</string>
   </void>
  </object>
 </void>
 <void index="4">
  <object class="predictions.Prediction">
   <void property="what">
    <string>Face to face client-server pricing structure will whiteboard robust communities.</string>
   </void>
   <void property="who">
    <string>River Wiza</string>
   </void>
  </object>
 </void>
 <void index="5">
  <object class="predictions.Prediction">
   <void property="what">
    <string>Future-proofed 5th generation protocol will strategize web-enabled networks.</string>
   </void>
   <void property="who">
    <string>Jarred Wehner</string>
   </void>
  </object>
```

```
</void>
<void index="6">
 <object class="predictions.Prediction">
  <void property="what">
   <string>Cross-group fresh-thinking encoding will reinvent dot-com systems.</string>
  </void>
  <void property="who">
   <string>Emily Roob</string>
  </void>
 </object>
</void>
<void index="7">
 <object class="predictions.Prediction">
  <void property="what">
   <string>Customizable assymetric database will visualize virtual action-items.</string>
  </void>
  <void property="who">
   <string>Elvis Ernser</string>
  </void>
 </object>
</void>
<void index="8">
 <object class="predictions.Prediction">
  <void property="what">
```

```
   <string>User-centric non-volatile open architecture will iterate world-class vortals.</string>
  </void>
  <void property="who">
   <string>Kathryn Hilpert</string>
  </void>
 </object>
</void>
<void index="9">
 <object class="predictions.Prediction">
  <void property="what">
   <string>Enhanced zero tolerance system engine will evolve turn-key deliverables.</string>
  </void>
  <void property="who">
   <string>Tanner Dietrich</string>
  </void>
 </object>
</void>
<void index="10">
 <object class="predictions.Prediction">
  <void property="what">
   <string>Distributed dynamic moratorium will iterate magnetic e-commerce.</string>
  </void>
  <void property="who">
   <string>Linnie Funk</string>
```

```
  </void>
 </object>
</void>
<void index="11">
 <object class="predictions.Prediction">
  <void property="what">
   <string>Synergistic demand-driven functionalities will visualize compelling vortals.</string>
  </void>
  <void property="who">
   <string>Emery Ward</string>
  </void>
 </object>
</void>
<void index="12">
 <object class="predictions.Prediction">
  <void property="what">
   <string>Robust intermediate extranet will facilitate best-of-breed synergies.</string>
  </void>
  <void property="who">
   <string>Craig Leuschke</string>
  </void>
 </object>
</void>
<void index="13">
```

```
 <object class="predictions.Prediction">
  <void property="what">
   <string>Digitized optimal conglomeration will exploit proactive relationships.</string>
  </void>
  <void property="who">
   <string>Shayna Lehner</string>
  </void>
 </object>
</void>
<void index="14">
 <object class="predictions.Prediction">
  <void property="what">
   <string>Universal fault-tolerant architecture will synthesize bleeding-edge channels.</string>
  </void>
  <void property="who">
   <string>Hollis McCullough</string>
  </void>
 </object>
</void>
<void index="15">
 <object class="predictions.Prediction">
  <void property="what">
   <string>Cloned assymetric intranet will enable innovative functionalities.</string>
  </void>
```

```
 <void property="who">
  <string>Mina Hayes</string>
 </void>
</object>
</void>
<void index="16">
 <object class="predictions.Prediction">
 <void property="what">
  <string>Decentralized 24/7 hub will target robust web-readiness.</string>
 </void>
 <void property="who">
  <string>River Friesen</string>
 </void>
</object>
</void>
<void index="17">
 <object class="predictions.Prediction">
 <void property="what">
  <string>Synergistic disintermediate policy will expedite back-end experiences.</string>
 </void>
 <void property="who">
  <string>Carmel Becker</string>
 </void>
</object>
```

```
</void>
<void index="18">
 <object class="predictions.Prediction">
  <void property="what">
   <string>Triple-buffered didactic emulation will visualize integrated channels.</string>
  </void>
  <void property="who">
   <string>Bartholome Walsh</string>
  </void>
 </object>
</void>
<void index="19">
 <object class="predictions.Prediction">
  <void property="what">
   <string>Configurable intangible alliance will scale sexy ROI.</string>
  </void>
  <void property="who">
   <string>Russel Robel</string>
  </void>
 </object>
</void>
<void index="20">
 <object class="predictions.Prediction">
  <void property="what">
```

```
   <string>Triple-buffered neutral collaboration will incubate B2B deliverables.</string>
  </void>
  <void property="who">
   <string>Charlene Mertz</string>
  </void>
 </object>
</void>
<void index="21">
 <object class="predictions.Prediction">
  <void property="what">
   <string>User-centric multi-state success will transform proactive convergence.</string>
  </void>
  <void property="who">
   <string>Letitia Pacocha</string>
  </void>
 </object>
</void>
<void index="22">
 <object class="predictions.Prediction">
  <void property="what">
   <string>Open-source multi-tasking time-frame will monetize rich partnerships.</string>
  </void>
  <void property="who">
   <string>Lottie Marks</string>
```

```
   </void>
  </object>
 </void>
 <void index="23">
  <object class="predictions.Prediction">
   <void property="what">
    <string>Optional static definition will unleash dynamic e-tailers.</string>
   </void>
   <void property="who">
    <string>Kaden Crona</string>
   </void>
  </object>
 </void>
 <void index="24">
  <object class="predictions.Prediction">
   <void property="what">
    <string>De-engineered systematic emulation will deploy out-of-the-box
partnerships.</string>
   </void>
   <void property="who">
    <string>Everardo Lind</string>
   </void>
  </object>
 </void>
```

```
 <void index="25">
  <object class="predictions.Prediction">
   <void property="what">
    <string>Synergistic 24/7 website will transition 24/7 methodologies.</string>
   </void>
   <void property="who">
    <string>Lilyan Thompson</string>
   </void>
  </object>
 </void>
 <void index="26">
  <object class="predictions.Prediction">
   <void property="what">
    <string>Reactive value-added middleware will engineer next-generation
partnerships.</string>
   </void>
   <void property="who">
    <string>Alessia O&amp;apos;Connell</string>
   </void>
  </object>
 </void>
 <void index="27">
  <object class="predictions.Prediction">
   <void property="what">
```

```
    <string>Self-enabling reciprocal synergy will generate seamless portals.</string>
   </void>
   <void property="who">
    <string>Reymundo Champlin</string>
   </void>
  </object>
 </void>
 <void index="28">
  <object class="predictions.Prediction">
   <void property="what">
    <string>Assimilated intermediate superstructure will drive vertical methodologies.</string>
   </void>
   <void property="who">
    <string>Immanuel Bergstrom</string>
   </void>
  </object>
 </void>
 <void index="29">
  <object class="predictions.Prediction">
   <void property="what">
    <string>Proactive demand-driven open architecture will innovate impactful
networks.</string>
   </void>
   <void property="who">
```

```
   <string>Dahlia Robel</string>
  </void>
 </object>
</void>
<void index="30">
 <object class="predictions.Prediction">
  <void property="what">
   <string>Balanced clear-thinking utilisation will expedite collaborative initiatives.</string>
  </void>
  <void property="who">
   <string>Deven Blanda</string>
  </void>
 </object>
</void>
<void index="31">
 <object class="predictions.Prediction">
  <void property="what">
   <string>Versatile tangible application will maximize rich e-business.</string>
  </void>
  <void property="who">
   <string>Hiram Gulgowski</string>
  </void>
 </object>
</void>
```

</array>

</java>

OR

ON THE BROWSER WINDOW

http://localhost:8080/appl/

THEN SEE THE FOLLOWING **OUTPUT**

Managed holistic contingency will grow killer action-items. Cornelius Tillman Vision-oriented zero administration time-frame will generate back-end interfaces. Conner Kulas Triple-buffered scalable function will productize visionary infomediaries. Loraine Ryan Reactive radical knowledge base will aggregate extensible vortals. Patricia Zulauf Face to face client-server pricing structure will whiteboard robust communities. River Wiza Future-proofed 5th generation protocol will strategize web-enabled networks. Jarred Wehner Cross-group fresh-thinking encoding will reinvent dot-com systems. Emily Roob Customizable assymetric database will visualize virtual action-items. Elvis Ernser User-centric non-volatile open architecture will iterate world-class vortals. Kathryn Hilpert Enhanced zero tolerance system engine will evolve turn-key deliverables. Tanner Dietrich Distributed dynamic moratorium will iterate magnetic e-commerce. Linnie Funk Synergistic demand-driven functionalities will visualize compelling vortals. Emery Ward Robust intermediate extranet will facilitate best-of-breed synergies. Craig Leuschke Digitized optimal conglomeration will exploit proactive relationships. Shayna Lehner Universal fault-tolerant architecture will synthesize bleeding-edge channels. Hollis McCullough Cloned assymetric intranet will enable innovative functionalities. Mina Hayes Decentralized 24/7 hub will target robust web-readiness. River Friesen Synergistic disintermediate policy will expedite back-end experiences. Carmel Becker Triple-buffered didactic emulation will visualize integrated channels. Bartholome Walsh Configurable intangible alliance will scale sexy ROI. Russel Robel Triple-buffered neutral collaboration will incubate B2B deliverables. Charlene Mertz User-centric multi-state success will transform proactive convergence. Letitia Pacocha Open-source multi-tasking time-frame will monetize rich partnerships. Lottie Marks Optional static definition will unleash dynamic e-tailers. Kaden Crona De-engineered systematic emulation will deploy out-of-the-box partnerships. Everardo Lind Synergistic 24/7 website will transition 24/7 methodologies. Lilyan Thompson Reactive value-added middleware will engineer next-generation partnerships. Alessia O'Connell Self-enabling reciprocal synergy will generate seamless portals. Reymundo Champlin Assimilated intermediate superstructure will drive vertical methodologies. Immanuel Bergstrom Proactive demand-driven open architecture will innovate impactful networks. Dahlia Robel Balanced clear-thinking utilisation will expedite

collaborative initiatives. Deven Blanda Versatile tangible application will maximize rich e-business. Hiram Gulgowski

EXPERIMENTS:

1. Write programs that implement the web services that involve database query. We wish to query the Parts table for all parts of a given vendor. The vendor will be a parameter to the Web service. The Web service will use a database connection to execute the SQL query. It will return an XML document as its result. The Web service will return parts and vendors only if the part ID is 1000 or greater: Use RESTful webservice in PHP.
2. Write java RESTful programs that implement the web services that involve creating simple REST API to perform POST or update and delete records.